CHAPTER 17

Cognito and WAF

User Pools · JWT · MFA · Social Login · WAF Rules · Bot Control · Identity Pool

AWS Tutorial · ShopFlow

17-00 Chapter Overview — Auth and Security at ShopFlow

ShopFlow auth stack: Cognito User Pool (user registration, login, MFA, JWT tokens) + Cognito Identity Pool (federated access to AWS services) + WAF WebACL (OWASP Top 10, rate limiting, bot control). The ALB validates JWTs from Cognito before requests reach ECS microservices — no auth code in the microservices themselves.

Layer	Service	ShopFlow Function	Integration Point
User Identity	Cognito User Pool	Customer registration, login, password reset, Google/Facebook OAuth	Mobile app + web frontend
JWT Tokens	Cognito User Pool	ID Token (user info), Access Token (API auth), Refresh Token (7-day session)	ALB Authenticate-Cognito action
ALB Auth	ALB Authenticate-Cognito action	Validates JWT on every request; adds x-amzn-oidc headers to request	ALB listener rule
API Protection	WAF WebACL on ALB	Rate limiting per IP, OWASP rules, bot control	ALB (all traffic)
AWS Resource Access	Cognito Identity Pool	Mobile app S3 upload, Kinesis events, signed URLs	STS AssumeRoleWithWebIdentity

ShopFlow Auth Flow

- Customer: signup or login via Cognito Hosted UI or mobile SDK
  - Cognito: validates credentials → issues JWT (ID + Access Token)
  - Optional: MFA via SMS or TOTP authenticator app
- Customer: sends request with Authorization: Bearer {access_token}
- ALB: validates JWT signature (Cognito JWKS) → Allow or 401
  - ALB adds x-amzn-oidc-identity (subject/user ID) header to request
- ECS microservice: reads x-amzn-oidc-identity header (no JWT parsing needed)

17-01 Cognito User Pool CDK

Cognito User Pool stores customer credentials and profile attributes. ShopFlow config: email as username, password policy (min 12 chars, upper+lower+number+symbol), optional MFA via TOTP, account verification via email link, self-service password reset. Lambda triggers for custom business logic (post-confirmation, pre-token generation).

User Pool CDK

```
const userPool = new cognito.UserPool(this,"ShopFlowUsers",{
userPoolName: "shopflow-customers",
signInAliases: {email: true, username: false},
selfSignUpEnabled: true,
accountRecovery: cognito.AccountRecovery.EMAIL_ONLY,
autoVerify: {email: true},
passwordPolicy: {
minLength: 12,
requireUppercase: true,
```

```
requireLowercase: true,
requireDigits: true,
requireSymbols: true,
tempPasswordValidity: Duration.days(3),
},
mfa: cognito.Mfa.OPTIONAL, // users can enable MFA voluntarily
mfaSecondFactor: {sms: true, otp: true}, // SMS or TOTP (Google Authenticator)
lambdaTriggers: {
postConfirmation: postConfirmLambda, // sync new user to Aurora users table
preTokenGeneration: tokenEnrichLambda, // add custom claims (tier, permissions)
},
removalPolicy: RemovalPolicy.RETAIN, // never delete User Pool — user data
deletionProtection: true,
});
```

17-02 Cognito App Client and Hosted UI

Cognito App Client is the OAuth 2.0 client configuration for the ShopFlow web and mobile apps. Separate app clients for web (client secret, server-side) and mobile (no client secret, PKCE flow). Hosted UI provides a customizable login page without building authentication UI from scratch.

App Client CDK

```
// Web app client (has client secret — for server-side auth code flow)
const webClient = userPool.addClient("WebAppClient",{
userPoolClientName: "shopflow-web",
generateSecret: true, // required for server-side OAuth code flow
authFlows: {
authorizationCodeGrant: true, // OAuth 2.0 Authorization Code + PKCE
userSrpAuth: true, // Secure Remote Password (SDK auth)
},
oAuth: {
flows: {authorizationCodeGrant: true},
scopes: [cognito.OAuthScope.OPENID, cognito.OAuthScope.EMAIL,
cognito.OAuthScope.PROFILE],
callbackUrls: ["https://shopflow.com/auth/callback"],
logoutUrls: ["https://shopflow.com/logout"],
},
accessTokenValidity: Duration.minutes(60),
idTokenValidity: Duration.minutes(60),
refreshTokenValidity: Duration.days(30),
preventUserExistenceErrors: true, // same error for wrong email and wrong password
});
```

Cognito Hosted UI CDK

```
const userPoolDomain = userPool.addDomain("ShopFlowDomain",{
customDomain: {
domainName: "auth.shopflow.com",
certificate: authCert, // ACM cert for auth.shopflow.com
},
});
// Hosted UI: https://auth.shopflow.com/login?client_id=xxx&response;_type=code&...
```

17-03 JWT Token Structure and Validation

Cognito issues three JWT tokens: ID Token (user profile claims), Access Token (API authorization scopes), Refresh Token (obtain new tokens without re-login). Tokens are signed with Cognito RSA private key; validated using Cognito JWKS (public keys). ALB validates Access Token signature and expiry on every request.

Token	Claims	Validity	Use
ID Token	sub, email, name, cognito:groups, custom:tier	60 min	User profile info — who is the user?
Access Token	sub, scope, cognito:groups, token_use=access	60 min	API authorization — what can the user do?
Refresh Token	username, device key	30 days	Exchange for new ID + Access tokens silently

JWT Claims — ShopFlow Access Token (decoded)

```
{
"sub": "a1b2c3d4-e5f6-7890-abcd-ef1234567890", // Cognito user ID
"cognito:groups": ["customers", "prime-members"],
"iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_POOLID",
"token_use": "access",
"scope": "openid email profile",
"aud": "3t4xxx-webclientid", // App Client ID
"exp": 1701234567, // Unix epoch (60 min from now)
"iat": 1701230967, // issued-at
"custom:tier": "prime", // custom attribute from Lambda trigger
"jti": "uuid-of-this-token" // JWT ID (for revocation tracking)
}
```

■ ALB validates the JWT on every request using Cognito's public JWKS endpoint. The ALB caches JWKS keys locally — no per-request HTTP call to Cognito. If Cognito rotates keys, the ALB refreshes its JWKS cache within 5 minutes. ECS microservices do NOT need to validate JWTs — they trust the x-amzn-oidc-identity header added by ALB after validation. This centralizes auth at the ALB boundary, eliminating JWT library dependencies from microservices.

17-04 ALB Cognito Authenticator Action

ALB Authenticate-Cognito action validates JWTs from Cognito for all incoming requests before routing to ECS. ShopFlow configures authenticateCognito on all listener rules that require authentication (/api/*). Public endpoints (/, /search, /products) use a separate listener rule without authentication.

ALB Cognito Auth CDK

```
// Authenticated routes: /api/* requires Cognito JWT
httpsListener.addAction("AuthenticatedApi",{
priority: 100,
conditions: [elbv2.ListenerCondition.pathPatterns(["/api/*"])],
action: elbv2.ListenerAction.authenticateCognito({
userPool,
userPoolClient: webClient,
userPoolDomain: userPoolDomain,
onUnauthenticatedRequest:
elbv2.UnauthenticatedAction.AUTHENTICATE, // redirect to Cognito login
sessionCookieName: "AWSELBAuthSessionCookie",
sessionTimeout: Duration.days(1),
next: elbv2.ListenerAction.forward([apiTargetGroup]),
}),
});
// Public routes: no auth (product browse, search)
httpsListener.addAction("PublicSearch",{
priority: 50, // evaluated first (lower number = higher priority)
conditions: [elbv2.ListenerCondition.pathPatterns(["/", "/search/*", "/products/*"])],
action: elbv2.ListenerAction.forward([productTargetGroup]), // no auth
});
```

ALB Auth Header	Value	ECS Usage
x-amzn-oidc-identity	Cognito sub (user UUID)	Extract userId for all requests — no JWT parsing
x-amzn-oidc-data	JWT payload (base64)	Decode for claims (email, groups) without signature verific

ALB Auth Header	Value	ECS Usage
x-amzn-oidc-access-token	Access token	Pass downstream if microservice needs to call another se
x-amzn-trace-id	X-Ray trace ID	Distributed tracing continuity

17-05 Cognito Lambda Triggers

Cognito Lambda Triggers execute custom code at specific points in the auth flow. ShopFlow uses: PostConfirmation (sync new user to Aurora users table), PreTokenGeneration (add custom claims: tier, permissions), CustomMessage (branded email templates), and PostAuthentication (last-login timestamp update).

PreTokenGeneration Lambda — Add Custom Claims

```
@Component
public class TokenEnricherHandler
implements RequestHandler<Map<String,Object>, Map<String,Object>> {
@Override
public Map<String,Object> handleRequest(Map<String,Object> event, Context ctx) {
String userId = (String) ((Map) event.get("request")).get("userAttributes").get("sub");
// Look up user tier from Aurora (or ElastiCache Redis)
UserProfile profile = userRepo.findByCognitoId(userId);
// Add custom claims to the token
Map<String,Object> resp = new HashMap<>(event);
Map<String,Object> claimsToAddOrOverride = Map.of(
"custom:tier", profile.getTier(), // "free" | "prime"
"custom:cartId", profile.getCartId(), // persistent cart UUID
"custom:permissions", profile.getPermissions() // "write:reviews,read:orders"
);
((Map) ((Map) resp.get("response")).get("claimsAndScopeOverrideDetails"))
.put("idTokenGeneration", Map.of("claimsToAddOrOverride", claimsToAddOrOverride));
return resp;
}
}
```

17-06 Social Login — Google and Facebook OAuth

Cognito supports social identity providers (IdPs): Google, Facebook, Apple Sign In. ShopFlow integrates Google OAuth2 for "Sign in with Google". Cognito handles the OAuth2 redirect, exchanges the Google authorization code for tokens, creates or links the Cognito user, and issues Cognito JWTs — the ShopFlow app only interacts with Cognito, not directly with Google.

Google IdP CDK

```
// Register Google OAuth2 in Cognito User Pool
const googleIdp = new cognito.UserPoolIdentityProviderGoogle(this,"Google",{
userPool,
clientId: googleClientId.stringValue,
clientSecret: googleClientSecret.secretValue.unsafeUnwrap(),
attributeMapping: {
email: cognito.ProviderAttribute.GOOGLE_EMAIL,
givenName: cognito.ProviderAttribute.GOOGLE_NAME,
profilePage: cognito.ProviderAttribute.GOOGLE_PICTURE,
// Map Google sub to Cognito custom attribute for linking:
custom: {
"custom:googleId": cognito.ProviderAttribute.GOOGLE_SUB,
}
},
scopes: ["email", "profile", "openid"],
});
userPool.registerIdentityProvider(googleIdp);
```

Social Auth Flow Step	Actor	Detail
1. User clicks Sign in with Google	Browser	Redirected to Cognito Hosted UI /oauth2/authorize?identity_provider=Google
2. Cognito redirects to Google	Cognito	GET accounts.google.com/o/oauth2/v2/auth with Cognito redirect_uri
3. User logs in to Google	Google/Browser	Google authenticates user; returns authorization code to Cognito callback
4. Cognito exchanges code for Google tokens	Cognito	POST to Google token endpoint; receives Google ID token + access token
5. Cognito maps attributes + issues its own JWT	Cognito	Creates Cognito user (or links existing); issues Cognito JWT tokens
6. Cognito returns JWT to ShopFlow app	Browser	ShopFlow receives Cognito tokens (not Google tokens) — unified auth

17-07 WAF WebACL — OWASP Rules and Rate Limiting

WAF WebACL protects ShopFlow from OWASP Top 10 attacks (SQL injection, XSS, LFI), bot traffic, and DDoS. WAF evaluates rules in priority order: rate limiting (first), AWS Managed Rules (second), custom rules (third). Requests blocked by WAF return HTTP 403 before reaching the ALB.

WAF WebACL CDK with Managed Rules

```
const wafAcl = new wafv2.CfnWebACL(this,"ShopFlowWaf",{
scope: "REGIONAL",
defaultAction: {allow: {}},
rules: [
// Rule 1: IP-based rate limiting (10K req per 5 min per IP)
{name:"IpRateLimit",priority:1,action:{block:{}},
statement:{rateBasedStatement:{limit:10000,aggregateKeyType:"IP"}},
visibilityConfig:{...}},
// Rule 2: AWS Managed Rules — Common Rule Set (OWASP Top 10)
{name:"AWSCommon",priority:2,statement:{managedRuleGroupStatement:{
vendorName:"AWS",name:"AWSManagedRulesCommonRuleSet"}},
overrideAction:{none:{}},visibilityConfig:{...}},
// Rule 3: AWS Managed Rules — Known Bad Inputs
{name:"BadInputs",priority:3,statement:{managedRuleGroupStatement:{
vendorName:"AWS",name:"AWSManagedRulesKnownBadInputsRuleSet"}},
overrideAction:{none:{}},visibilityConfig:{...}},
// Rule 4: Bot Control (blocks scrapers, curl-based bots)
{name:"BotControl",priority:4,statement:{managedRuleGroupStatement:{
vendorName:"AWS",name:"AWSManagedRulesBotControlRuleSet",
managedRuleGroupConfigs:[{awsManagedRulesBotControlRuleSet:{
inspectionLevel:"TARGETED"}}]}},
overrideAction:{none:{}},visibilityConfig:{...}},
// Rule 5: Custom — block countries under export controls
{name:"GeoBlock",priority:5,action:{block:{}},
statement:{geoMatchStatement:{countryCodes:["KP","IR","CU","SY"]}},
visibilityConfig:{...}},
],
visibilityConfig:{cloudWatchMetricsEnabled:true,metricName:"ShopFlowWAF",
sampledRequestsEnabled:true},
});
```

17-08 WAF Custom Rules and Logging

WAF custom rules use regex, string match, and JSON body inspection. ShopFlow custom rules: block requests with SQL keywords in query parameters (defense-in-depth alongside common rules), block User-Agent headers matching known scraper signatures, and rate-limit the /api/checkout endpoint separately (stricter: 100 req/5min per IP).

Custom Rule	Match Condition	Action	Purpose
Checkout rate limit	/api/checkout* path + rate > 100/5min per IP	BLOCK (429)	Prevent brute-force checkout abuse

Custom Rule	Match Condition	Action	Purpose
Scraper user agent	User-Agent contains: Scrapy, python-requests, curl, Http-client	COUNT → review → BLOCK	Allow initially; monitor false positives before
Large body block	Request body > 10MB	BLOCK	Prevent oversized payload attacks on /api/p
Admin path geo-restrict	/admin/* path from non-US IP	BLOCK	Admin console accessible from US only

WAF Logging CDK

```
// WAF Logs → Kinesis Firehose → S3 for analysis
const wafLogGroup = new logs.LogGroup(this,"WafLogs",{
logGroupName: "aws-waf-logs-shopflow", // MUST start with "aws-waf-logs-"
retention: logs.RetentionDays.ONE_MONTH,
});
new wafv2.CfnLoggingConfiguration(this,"WafLogging",{
resourceArn: wafAcl.attrArn,
logDestinationConfigs: [wafLogGroup.logGroupArn],
redactedFields: [ // redact sensitive header values from WAF logs
{singleHeader: {name: "authorization"}},
{singleHeader: {name: "cookie"}},
],
});
```

■ WAF Bot Control with TARGETED inspection level uses signal correlation across requests (IP, session, browser fingerprint) to detect sophisticated bots that rotate IPs and mimic human behavior. Common (basic) inspection only checks User-Agent and request patterns. ShopFlow uses TARGETED for Black Friday: prevents price scrapers from monitoring flash sale prices and programmatically adding items. TARGETED inspection costs: $1.00/million requests (10x Common inspection at $0.10/million).

17-09 Cognito Identity Pool — Temporary AWS Credentials

Cognito Identity Pool provides temporary AWS credentials to authenticated customers. ShopFlow uses Identity Pool for: mobile app uploading product review images directly to S3 (no pre-signed URL server needed), and for authenticated Kinesis event streaming (behavior tracking). Credentials expire in 1 hour; Cognito SDK auto-refreshes using the Cognito Refresh Token.

Identity Pool CDK

```
const identityPool = new cognito.CfnIdentityPool(this,"ShopFlowIdentityPool",{
identityPoolName: "shopflow_customers",
allowUnauthenticatedIdentities: false,
cognitoIdentityProviders: [{
clientId: webClient.userPoolClientId,
providerName: userPool.userPoolProviderName,
serverSideTokenCheck: true,
}],
});
// IAM role for authenticated customers
const authenticatedRole = new iam.Role(this,"AuthenticatedRole",{
assumedBy: new iam.FederatedPrincipal("cognito-identity.amazonaws.com",{
StringEquals: {"cognito-identity.amazonaws.com:aud": identityPool.ref},
"ForAnyValue:StringLike":{"cognito-identity.amazonaws.com:amr":"authenticated"},
},"sts:AssumeRoleWithWebIdentity"),
});
// Allow customer to upload to their own S3 "folder" only:
authenticatedRole.addToPolicy(new iam.PolicyStatement({
actions: ["s3:PutObject"],
resources: [`${reviewsBucket.bucketArn}/\${cognito-identity.amazonaws.com:sub}/*`],
}));
```

17-10 MFA Implementation — TOTP and SMS

ShopFlow supports two MFA methods: TOTP (Time-based One-Time Password via Google Authenticator) and SMS OTP. TOTP is preferred: no per-message cost, works offline, more secure (not SIM-swappable). ShopFlow encourages TOTP by showing a "Recommended" label in the MFA setup UI. SMS MFA requires SNS for delivery.

MFA Method	How It Works	Cost	Security Level	ShopFlow
TOTP (Google Auth, Authy)	User scans QR code once; enters 6-digit code that rotates every 30s	Free	High — codes generated offline, not intercepted	Recommended (shown first in UI)
SMS OTP	One-time code sent via SNS SMS to phone number	$0.00645/SMS (US)	Medium — SIM swap attack vulnerable	Optional fallback for non-tech us
Email OTP (Cognito feature)	Code sent to verified email address	SES cost (~$0)	Medium — requires email access	Not used (users already verified

TOTP MFA Setup Flow — Spring Boot

```
// Step 1: Start TOTP association (get secret key for QR code)
AssociateSoftwareTokenRequest req = AssociateSoftwareTokenRequest.builder()
.accessToken(userAccessToken).build();
AssociateSoftwareTokenResponse resp = cognitoClient.associateSoftwareToken(req);
String secretCode = resp.secretCode();
// Build QR code URI:
otpauth://totp/ShopFlow:user@email.com?secret=BASE32SECRET&issuer;=ShopFlow
// Display as QR code image in browser (user scans with Google Authenticator)
// Step 2: Verify TOTP (user enters 6-digit code from app)
VerifySoftwareTokenResponse verifyResp = cognitoClient.verifySoftwareToken(
VerifySoftwareTokenRequest.builder()
.accessToken(userAccessToken).userCode(totpCode).build());
// After verification: TOTP MFA is enabled for this user
```

17-11 Cognito Advanced Security — Adaptive Authentication

Cognito Advanced Security (ACSS) adds risk-based authentication: detects compromised credentials, unusual sign-in patterns (new device, new location, impossible travel), and bot sign-in attempts. ShopFlow enables ACSS in AUDIT mode (log all events) for 2 weeks then switches to ENFORCED (block high-risk sign-ins, require MFA for medium risk).

Cognito Advanced Security CDK

```
const userPool = new cognito.UserPool(this,"ShopFlowUsers",{
// ... other config ...
advancedSecurityMode: cognito.AdvancedSecurityMode.ENFORCED,
// AUDIT: log only (no enforcement)
// ENFORCED: block or require MFA based on risk score
});
```

Risk Level	ACSS Action (ENFORCED)	Example Trigger	ShopFlow Response
Low risk	Allow sign-in normally	Normal login from usual device/location	No friction — user logs in as normal
Medium risk	Require MFA even if MFA is optional	New device, unusual location for this user	User prompted for TOTP or SMS code
High risk	Block sign-in; notify user via email	Credential stuffing attempt; leaked password; impossible travel	Sign-in blocked; security email sent to use
Compromised credentials	Block sign-in; force password reset	Password found in HaveIBeenPwned database	User forced to reset password before acc

■ Cognito Advanced Security is an additional cost: $0.05 per monthly active user (MAU). ShopFlow: 1M MAU × $0.05 = $50,000/year — expensive! Start with AUDIT mode for 2-4 weeks to baseline the risk score distribution and false positive rate. Many e-commerce sites with geographically diverse customer base have high medium-risk event rates because customers legitimately travel and shop from different locations. Tune before ENFORCED to avoid locking out legitimate customers.

17-12 WAF Rate Limiting Per User vs Per IP

IP-based rate limiting is the default but insufficient for users behind corporate NAT or shared IPv4 addresses (CDN, office NAT). User-based rate limiting (rate by HTTP header value) limits per-user API abuse even when multiple users share an IP. ShopFlow: Checkout endpoint rate-limited by x-amzn-oidc-identity (per user), not by IP.

WAF User-Based Rate Limit Rule

```
{
"name": "CheckoutPerUserRateLimit",
"priority": 5,
"action": {"block": {}},
"statement": {
"rateBasedStatement": {
"limit": 10, // max 10 checkout attempts per 5 minutes per user
"aggregateKeyType": "CUSTOM_KEYS",
"customKeys": [{
"header": {
"name": "x-amzn-oidc-identity", // Cognito user ID added by ALB
"textTransformations": [{"priority":0,"type":"NONE"}]
}
}],
"scopeDownStatement": {
"byteMatchStatement": {
"searchString": "/api/checkout",
"fieldToMatch": {"uriPath": {}},
"textTransformations":[{"priority":0,"type":"LOWERCASE"}],
"positionalConstraint": "STARTS_WITH"
}
}
}
}
}
```

Rate Limit Key	Scope	Best For	Limitation
IP address (default)	All requests from one IPv4 or IPv6	Bot scraping, DDoS from single IP	Fails for shared IP (NAT, CDN: all users sh
HTTP Header (x-forwarded-for)	Requests with matching header value	API abuse by specific clients	Header can be spoofed; must use ALB-add
Cognito user ID (x-amzn-oidc-identity)	Authenticated requests per user	Per-user checkout abuse, payment fraud attempts	Only works for authenticated endpoints (re
Query string value	Requests with matching query param	Partner API key abuse (rate by apiKey param)	Param must be present in all requests

17-13 Secrets Rotation — Cognito App Client Secret

Cognito App Client secrets must be rotated periodically for compliance. ShopFlow automates rotation: Secrets Manager rotates the Cognito app client secret, updates SSM Parameter Store, and sends an SNS notification for ECS services to pick up the new secret on next restart. Zero-downtime rotation uses a rotation window.

Secret	Rotation Frequency	Rotation Mechanism	Zero-Downtime Strategy
Cognito App Client Secret	90 days (compliance)	Lambda: call Cognito CreateUserPoolClient with new secret, update SSM, notify ECS	Old + new secret valid during 30-min
Aurora DB password	30 days	Secrets Manager built-in Aurora rotation Lambda	Aurora supports multiple passwords; old p
JWT signing key (RSA)	Managed by Cognito	Cognito rotates automatically; multiple JWKS keys active simultaneously	JWKS has multiple keys; clients use kid (k
WAF IP set	Real-time (threat intel feed)	Lambda triggered by threat intel SNS; updates WAF IP set via API	Atomic WAF IP set update; no request int

17-14 WAF Shield Advanced and DDoS Protection

AWS Shield Standard (free) provides basic DDoS protection at the network and transport layer. Shield Advanced ($3,000/month base) adds Layer 7 DDoS protection, DRT (DDoS Response Team) 24/7 access, cost protection (credits for scale-out costs during DDoS), and real-time DDoS visibility in CloudWatch. ShopFlow evaluates Shield Advanced for Black Friday.

Shield Tier	Cost	DDoS Protection	ShopFlow Recommendation
Shield Standard (default)	Free (included)	Layer 3/4 (SYN floods, UDP reflection) — automatic	Use always — no configuration needed
WAF + Shield Standard (ShopFlow default)	WAF costs only	Layer 3/4 + Layer 7 (WAF rules block HTTP floods)	Use for standard operations
Shield Advanced	$3,000/month + data transfer fees	Layer 3/4/7 + DRT + cost protection + real-time metrics	Consider for Black Friday (2-day cost: $200)
Shield Advanced + DRT engagement	$3,000/month + DRT fees	DRT actively mitigates during attack (faster than self-service)	Recommended if revenue during Black Friday

WAF Block Rules as First DDoS Defense

```
// WAF is the primary Layer 7 DDoS defense (without Shield Advanced):
// Rule 1: IP rate limit 10K/5min — blocks HTTP floods from single IPs
// Rule 2: AWSManagedRulesBotControlRuleSet — blocks known botnet traffic
// Rule 3: AWSManagedRulesAmazonIpReputationList — blocks DDoS botnet IPs
// Rule 4: AWSManagedRulesAnonymousIpList — blocks Tor exit nodes, VPNs
// These 4 rules handle 99% of common DDoS attacks without Shield Advanced.
// For volumetric DDoS (10 Gbps+): Shield Advanced or CloudFront (absorbs).
```

17-15 Federated Identity for Internal Tools

ShopFlow internal admin panel (inventory management, order management, analytics) uses SAML 2.0 federation with the company Active Directory via AWS IAM Identity Center (SSO). Engineers log in with their corporate credentials — no separate AWS username. IAM Identity Center maps AD groups to IAM roles (read-only for analysts, full access for admins).

Auth Method	Identity Source	Tokens Issued	ShopFlow Usage
Customer auth (Cognito User Pool)	Cognito username/password or Google OAuth	Cognito JWT (ID + Access + Refresh)	shopflow.com customer-facing endpoints
Internal admin auth (IAM Identity Center)	Corporate Active Directory (SAML 2.0)	AWS temporary credentials (STS)	admin.shopflow.com + AWS Console access
API-to-API auth	IAM role (ECS task role)	STS temporary credentials (auto-rotated)	ECS microservice calling Aurora, S3, SQS
Mobile S3 upload (Identity Pool)	Cognito User Pool JWT exchanged for AWS creds	STS temporary credentials (1-hour TTL)	Customer uploading product review images

17-16 WAF False Positive Management

WAF false positives (legitimate traffic blocked by WAF rules) are common when enabling managed rules without testing. ShopFlow process: (1) Start rules in COUNT mode, (2) Analyze WAF logs for legitimate requests that matched rules, (3) Create rule exclusions for false positives, (4) Switch to BLOCK mode. Ongoing: weekly review of WAF metrics.

WAF Log Analysis for False Positives

```
-- Athena query: find WAF-blocked requests by non-bot user agents
SELECT COUNT(*) as blocked,
http_request.clientip,
http_request.headers[array_position(http_request.headers,
filter(http_request.headers, x -> x.name = User-Agent)[1])] as ua,
terminatingruleid
FROM waf_logs
WHERE action = BLOCK
AND from_unixtime(timestamp/1000) > CURRENT_TIMESTAMP - INTERVAL 24 HOUR
GROUP BY http_request.clientip, ua, terminatingruleid
ORDER BY blocked DESC LIMIT 50
```

WAF Rule Exclusion Type	When to Use	CDK Syntax	Risk
ExcludeRule (skip specific rule)	One managed rule causes too many false positives	excludedRules: [{name:"SQLi_BODY"}]	Disables the rule entirely for all r
RuleGroupReferenceStatement override	Keep rule group but change action for specific rule	ruleActionOverrides: [{name:"...",actionToUse:{count:{}}}]	Counts but does not block — stil
Scope-down statement	Apply rule only to specific paths or IPs	scopeDownStatement: {byteMatchStatement:{...}}	Most precise — rule applies only

17-17 Session Management and Token Refresh

ShopFlow web frontend (React SPA) stores tokens in memory (not localStorage). Access Token stored in memory expires in 60 minutes. The frontend uses Cognito SDK to silently refresh the access token using the Refresh Token (stored in HttpOnly cookie — not accessible to JavaScript). Silent refresh happens automatically before token expiry.

Token Storage	XSS Risk	CSRF Risk	Token Exposure
localStorage (do NOT use)	High — any JS can read localStorage	Low (CORS protects API calls)	Persistent token exposed to any JS on page
In-memory variable (ShopFlow)	Low — XSS cannot read other JS variables	Low	Access token lost on page refresh; SPA manages refresh
HttpOnly Cookie (Refresh Token)	None — cookie not readable by JS	Medium — CSRF token required for refresh endpoint	Refresh token sent automatically by browser sends automatically
sessionStorage	Medium — same tab only	Low	Token lost on tab close; slight improvement over localStorage

Cognito SDK Silent Token Refresh

```
// React: Amplify SDK auto-refreshes access token
import { Auth } from "aws-amplify";
// Configure Amplify (once, at app startup):
Amplify.configure({
Auth: {
region: "us-east-1",
userPoolId: "us-east-1_XXXXXXXX",
userPoolWebClientId: "XXXXX",
oauth: {domain: "auth.shopflow.com", ...}
}
});
// Get current session (Amplify refreshes automatically if needed):
const session = await Auth.currentSession();
const accessToken = session.getAccessToken().getJwtToken();
// Amplify SDK: if access token expires within 5 min, silently calls
// Cognito /oauth2/token with refresh_token to get new tokens
```

17-18 Cognito User Pool Groups and RBAC

Cognito User Pool Groups enable role-based access control (RBAC). ShopFlow groups: customers (all users), prime-members (paid subscribers), sellers (marketplace vendors), admins (internal staff). Groups appear in the cognito:groups JWT claim. ECS services check group membership for feature gating without querying a database.

User Pool Groups CDK

```
// Create groups in User Pool
new cognito.CfnUserPoolGroup(this,"CustomersGroup",{
userPoolId: userPool.userPoolId,
groupName: "customers",
description: "All registered customers",
precedence: 10, // lower = higher priority for multi-group users
});
new cognito.CfnUserPoolGroup(this,"PrimeMembersGroup",{
userPoolId: userPool.userPoolId,
groupName: "prime-members",
description: "Paid Prime subscribers",
precedence: 5,
```

```
});
```

RBAC in ECS Service — Check Cognito Group

```
@RestController
public class ProductController {
@GetMapping("/api/products/{id}/prime-price")
public PrimePrice getPrimePrice(@RequestHeader("x-amzn-oidc-data") String oidcData,
@PathVariable String id) {
// Decode OIDC data header (base64 JWT payload) — no signature verification needed
// (ALB already verified the full JWT; this header is trusted)
Map<String,Object> claims = decodeBase64Jwt(oidcData);
List<String> groups = (List<String>) claims.get("cognito:groups");
if (!groups.contains("prime-members")) {
throw new AccessDeniedException("Prime membership required");
}
return priceService.getPrimePrice(id);
}
}
```

17-19 Account Takeover Prevention

ShopFlow implements layered account takeover (ATO) prevention: Cognito Advanced Security (risk-based MFA), WAF rate limiting on /auth endpoints, Secrets Manager for password reset tokens, and CloudWatch monitoring for unusual account activity (multiple failed logins, rapid geolocation changes).

ATO Vector	Detection Method	Prevention	ShopFlow Implementation
Credential stuffing (leaked passwords)	Cognito ACSS: compromised credential check	Block sign-in, force password reset	ACSS ENFORCED mode: blocks auto-reset
Brute force login	WAF rate limit: 10 failed logins/5 min per IP	CAPTCHA after 5 fails; block after 10	WAF rule on /oauth2/token endpoint
Password reset token theft	Secure token: 32-byte random, 15-min TTL, single-use	Token invalidated on first use	Cognito built-in (verification codes)
Session hijacking	Short Access Token TTL (60 min)	Token expiry limits exposure window	60-min access token + 30-day refresh token
Impossible travel	Cognito ACSS: location risk scoring	Require MFA when risk=medium	ACSS ENFORCED: MFA on new location

17-20 WAF and Cognito Metrics Dashboard

ShopFlow Security Dashboard in CloudWatch tracks WAF and Cognito metrics together. Key metrics: WAF blocked requests per rule (which rules fire most), Cognito sign-in success/failure rate, WAF allowed vs blocked ratio, and bot traffic percentage. Weekly security review of these metrics.

Metric	Namespace	Widget Type	Alert Threshold
WAF BlockedRequests	AWS/WAFV2	Time series + rule breakdown	>1000 blocks/min (potential attack)
WAF AllowedRequests	AWS/WAFV2	Time series	Drop >20% (WAF over-blocking or traffic drop)
Cognito SignInSuccesses	AWS/Cognito	Time series	Drop >20% from baseline (auth issues)
Cognito TokenRefreshSuccesses	AWS/Cognito	Time series	Drop >10% from baseline (session issues)
Cognito SignUpSuccesses	AWS/Cognito	Count per hour	Spike >10x baseline (bot signup flood)
Cognito AccountTakeoverRisk (ACSS)	AWS/Cognito	Count by risk level	Any HIGH risk = immediate alert

Security Dashboard CDK

```
const secDashboard = new cloudwatch.Dashboard(this,"SecurityDashboard",{
dashboardName: "ShopFlow-Security",
});
secDashboard.addWidgets(
```

```
new cloudwatch.GraphWidget({
title: "WAF: Blocked vs Allowed Requests",
left: [new cloudwatch.Metric({namespace:"AWS/WAFV2",metricName:"BlockedRequests",
dimensions:{WebACL:"ShopFlowWAF",Region:"us-east-1",Rule:"ALL"},
period:Duration.minutes(1),statistic:"Sum"})],
right:[new cloudwatch.Metric({namespace:"AWS/WAFV2",metricName:"AllowedRequests",
dimensions:{WebACL:"ShopFlowWAF",Region:"us-east-1",Rule:"ALL"},
period:Duration.minutes(1),statistic:"Sum"})],
width: 24, height: 6,
}),
);
```

17-21 Cognito Hosted UI Customization

Cognito Hosted UI can be customized with ShopFlow branding: logo, background color, CSS overrides. The hosted UI supports custom domains (auth.shopflow.com). For full UI control, ShopFlow uses the Cognito SDK directly in the React frontend (custom UI components, no hosted UI redirect) for the login page.

Approach	Customization Level	Implementation Effort	ShopFlow Choice
Cognito Hosted UI (default)	Logo + background color + limited CSS	Low — configure in Cognito console	Used for admin/internal tools
Cognito Hosted UI + custom domain	Same + custom domain (auth.shopflow.com)	Low + DNS + ACM cert	Used for admin tools on auth.shopflow.com
Custom UI with Cognito SDK (Amplify)	Full control — any design	High — build all auth forms in React	Used for shopflow.com customer login
Custom UI with Cognito REST API directly	Full control + no AWS SDK dependency	Very high — implement SRP auth flow manually	Not used — Amplify SDK preferred

Hosted UI CSS Customization CDK

```
// Apply ShopFlow branding to Cognito Hosted UI
new cognito.CfnUserPoolUICustomizationAttachment(this,"HostedUiTheme",{
userPoolId: userPool.userPoolId,
clientId: "ALL", // applies to all app clients
css: `
.logo-querystring-parameter { display: block; }
.banner-customizable { background-color: #232f3e; } /* AWS dark navy */
.submitButton-customizable {
background-color: #ff9900; /* AWS orange */
color: white;
}
.label-customizable { font-weight: 600; color: #232f3e; }
\`,
});
```

■ For the customer-facing login flow, prefer a custom UI with the Amplify SDK over the Cognito Hosted UI. The hosted UI has limitations: fixed layout, limited CSS overrides, redirect-based flow (user leaves shopflow.com temporarily). Custom UI: stays on shopflow.com throughout; full design control; can add inline error messaging and social login buttons. The hosted UI is excellent for admin tools and internal applications where UI polish is less critical.

17-22 Cross-Origin Resource Sharing and Security Headers

ShopFlow API requires CORS configuration to allow the React SPA (shopflow.com) to call API endpoints (api.shopflow.com). Strict CORS policy: only shopflow.com origin allowed; explicit methods; credentials allowed (for cookies). Security headers added by CloudFront Function prevent clickjacking, XSS, and MIME sniffing.

Security Header	Value	Attack Prevented
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Downgrade attack (HTTP redirect exploit)
Content-Security-Policy	default-src self; script-src self cdn.shopflow.com	XSS: only scripts from shopflow.com and CDN allowed
X-Frame-Options	DENY	Clickjacking: page cannot be embedded in iframes

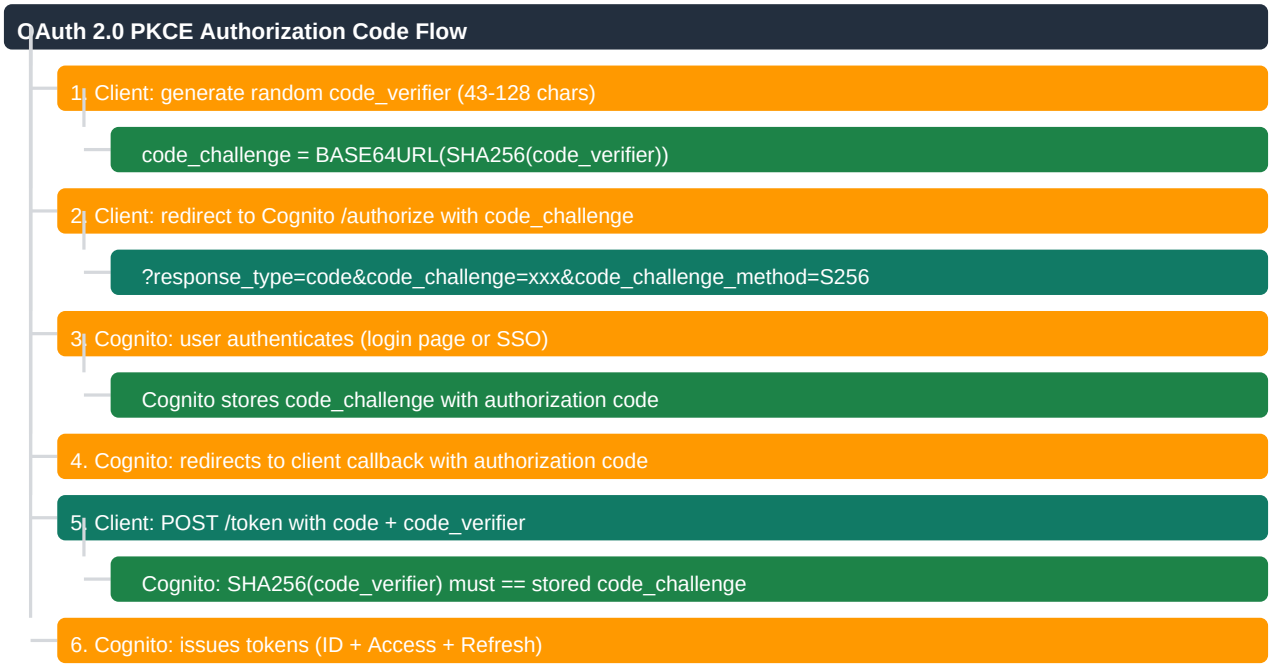
Security Header	Value	Attack Prevented
X-Content-Type-Options	nosniff	MIME type sniffing attack prevention
Referrer-Policy	strict-origin-when-cross-origin	Prevent referrer leakage to third parties
Permissions-Policy	camera=(), microphone=(), geolocation=()	Disable unnecessary browser permissions

CloudFront Function for Security Headers

```
// CloudFront Function (Viewer Response): add security headers to all responses
function handler(event) {
var response = event.response;
var headers = response.headers;
headers["strict-transport-security"] = {
value: "max-age=31536000; includeSubDomains; preload"};
headers["x-frame-options"] = {value: "DENY"};
headers["x-content-type-options"] = {value: "nosniff"};
headers["referrer-policy"] = {value: "strict-origin-when-cross-origin"};
headers["permissions-policy"] = {value: "camera=(), microphone=()"},
return response;
}
```

17-23 PKCE Flow for Mobile and SPA Authentication

PKCE (Proof Key for Code Exchange) is the OAuth 2.0 extension for public clients (mobile apps, SPAs) that cannot safely store a client secret. ShopFlow React SPA and iOS/Android apps use PKCE: the client generates a code_verifier, hashes it to code_challenge, and sends the challenge with the auth request. Without the original verifier, a stolen auth code is useless.



Security Property	Without PKCE	With PKCE
Stolen auth code (man-in-middle)	Attacker exchanges code for tokens	Code useless without code_verifier (stored only in legitimate client
Client secret required	Yes (for server-side apps)	No — public clients (SPA, mobile) safe without secret
Replay attack protection	Authorization code single-use (standard OAuth)	PKCE adds additional layer: verifier also single-use

■ Cognito supports PKCE for all app clients with authorizationCodeGrant: true. Set generateSecret: false for public clients (SPA, mobile) that cannot securely store a secret. generateSecret: true is only for server-side apps where the secret is stored server-side (never in browser or mobile app binary). If a secret is embedded in a mobile app, it can be extracted by decompiling the APK — never use client secrets in mobile apps.

17-24 WAF Managed Rule Groups — Cost and Coverage

AWS Managed Rule Groups are pre-built WAF rule sets maintained by AWS threat intelligence. ShopFlow evaluates each managed rule group for cost vs coverage before enabling. Some rules have high false-positive rates for e-commerce traffic and need COUNT mode testing. Bot Control TARGETED inspection is the most expensive but most effective for Black Friday scraper prevention.

Managed Rule Group	Rules Count	Cost (per 1M req)	ShopFlow Status	False Positive Risk
AWSManagedRulesCommonRuleSet (CRS)	~35 rules	$1.00/million	Enabled (BLOCK)	Medium — some rules block curl/
AWSManagedRulesKnownBadInputsRuleSet	~8 rules	$1.00/million	Enabled (BLOCK)	Low — targets log4j, SSRF, path
AWSManagedRulesBotControlRuleSet (common)	~30 rules	$1.00/million	Enabled (BLOCK)	Medium — blocks some legitimate
AWSManagedRulesBotControlRuleSet (TARGETED)	~40 rules	$10.00/million (10x!)	Enabled during Black Friday only	Low (targeted behavioral analysis
AWSManagedRulesAmazonIpReputationList	~5 rule groups	$1.00/million	Enabled (BLOCK)	Very low — known malicious IPs f
AWSManagedRulesSQLiRuleSet	~6 rules	$1.00/million	Not enabled (CRS covers SQLi)	Would duplicate CRS; unnecessa

Total WAF Cost Estimate

```
// Normal operation: 1B requests/month (1000 req/s x 86400 x 30)
// WAF WebACL base: $5.00/month
// Rules (4 managed rule groups): 4 x $1.00 per 1M requests
// = 4 x $1.00 x 1000 = $4,000/month at 1B requests
// Black Friday (3 days): 10,000 req/s x 86400 x 3 = 2.6B requests
// + TARGETED Bot Control ($10/M): $10 x 2600 = $26,000 (3 days)
// Tip: use scope-down statement to apply TARGETED only to /search and /products
// scope-down reduces evaluated requests by 60%; cost drops proportionally
```

■ Scope-down statements dramatically reduce WAF cost for expensive rule groups. Without scope-down, Bot Control TARGETED evaluates EVERY request including /actuator/health (1000 req/s × 30 = 2.6B/month). With scope-down to /search/* and /products/*: only ~30% of requests (product browse + search). Cost: $10 × 780M = $7,800/month vs $26,000/month without scope-down. Always add scope-down to expensive rule groups.

17-BP Best Practices and Anti-Patterns

✓ Use ALB Authenticate-Cognito action for JWT validation — centralize auth at the load balancer; microservices trust x-amzn-oidc-identity header

✓ Enable preventUserExistenceErrors on App Client — same error message for wrong email and wrong password prevents user enumeration attacks

✓ Use TOTP MFA for privileged accounts (admin, finance) — more secure than SMS; not vulnerable to SIM-swap attacks

✓ Use Refresh Token rotation — after each use of a refresh token, Cognito issues a new refresh token and invalidates the old one (reuse detection)

✓ Add x-amzn-oidc-data and x-amzn-oidc-identity to WAF allowed headers — WAF custom header inspection should whitelist these ALB-added headers

✓ WAF rule testing: start all custom rules in COUNT mode — observe logs for 1-2 weeks before switching to BLOCK; prevents false positives from blocking legitimate customers

✓ Enable WAF logging and redact Authorization and Cookie headers — logs contain enough context for threat analysis without exposing sensitive tokens

✓ Use Cognito User Pool groups for role-based access — groups appear in cognito:groups JWT claim; check group membership in ECS service for admin functions

✓ Set short Access Token validity (60 min) and longer Refresh Token validity (30 days) — stolen access tokens expire quickly; users stay logged in via refresh token

✓ Separate User Pool for internal admin users — different password policy, mandatory MFA, stricter IP restriction via WAF geographic rule

■ Never validate JWTs in microservices if ALB handles it — double validation wastes CPU; trust x-amzn-oidc-identity from ALB (it only forwards after successful JWT verification)

■ Do not use USER_PASSWORD_AUTH flow in mobile apps — sends password in cleartext over HTTPS to Cognito; use USER_SRP_AUTH (Secure Remote Password) which never sends the password

■ Avoid storing custom attributes in Cognito that change frequently — User Pool custom attributes are immutable per user once set; use Lambda trigger to read from Aurora/Redis instead

■ Do not enable ALLOW_USER_PASSWORD_AUTH on the App Client unless explicitly required — prefer SRP or OAuth2 PKCE flows

17-QR Quick Reference and Interview Q&A

Concept	Key Value / Rule
JWT vs session cookie	JWT: stateless; server does not store session; validated by signature (no DB lookup). Session cookie: ser
Cognito User Pool vs Identity Pool	User Pool: username/password/OAuth identity store. Issues JWTs (ID + Access + Refresh tokens). For AI
WAF rule action: COUNT vs BLOCK	COUNT: evaluates rule, logs match to CloudWatch, but ALLOWS the request through. Used for testing ne
Cognito token claims	ID Token: who the user is (email, name, custom:tier). Use for displaying user profile in UI. Access Token:
WAF Bot Control vs IP reputation list	IP Reputation List (AWSManagedRulesAmazonIpReputationList): blocks known malicious IPs (Tor exit no
Cognito Lambda trigger timeout	All Cognito Lambda triggers have a 5-second timeout. If the trigger Lambda exceeds 5s (e.g., slow Aurora